Worker event-loop blocking — profile evidence (Phase 1)

Branch: `fix/worker-isolation` (base `35884ee`). Context: batch 1274 (event-loop p99 lag 36–242 s, RSS ~1.2 GB,
PG pool acquire timeouts with the usage pool, max=2, failing first, Redis ping RTT 7–41 s, BullMQ `since_renew` ≈ 1,250,909 ms
against a 300,000 ms lock, “could not renew lock”, stalled redelivery, attempt 2 with `last_error=NULL`).

Question: where does the worker main thread spend its synchronous (blocking) CPU time while running real scoring jobs?
And is that time **CONCENTRATED** in a few functions (fix: move those into a `worker_threads` pool) or **DISTRIBUTED**
across the pipeline (fix: move the whole per-job processor off the main loop)?

Verdict: DISTRIBUTED. Details in §4.

---

1. Method

The harness is `server/scripts/profile-worker-blocking.ts`. It runs the **real** `runAnalysisPipeline({ skipFetch: true })`
(corpus hygiene → boilerplate → summarise → chunk/BM25 → aboutness → temporal → evidence packs → scoring → score-write)
on real saved production corpora. Everything external is stubbed:

Concern	How it was handled
Network	A zero-network guard patches `http`, `https`, `net` and `fetch` to throw. **1** attempt was blocked: `https.request www.sec.gov` (CIK resolution). No real provider or LLM call was made.
LLM providers	12 stub providers with **400 ms async latency** each (`PROFILE_PROVIDER_LATENCY_MS`). Every second call answers “Yes” with a verbatim quote taken from the prompt, so the evidence and verification paths are exercised.
LLM permits	The real `LLM_MAX_CONCURRENCY=8` semaphore.
Postgres	A `pg` pool stub. The corpus query (`FROM documents d LEFT JOIN document_content`) is served from `/home/ubuntu/var_corpus/<id>.json`. Every other query returns 0 rows.
Corpora	Production corpora 12, 11, 18 and 14: 166 / 177 / 175 / 99 docs, 43.0 / 33.3 / 23.0 / 20.2 M chars.
Framework	29 real measures from `var_measures.json`, padded to **36** (`-dupN` ids) to match production.
Concurrency	`PROFILE_CONCURRENCY=4` (= prod `WORKER_CONCURRENCY`), and a single-job control run with concurrency 1.
Lifecycle	Each company runs inside `runWithDiagContext` with a real attempt lifecycle (the same as `worker.ts`).
Instruments	A 50 ms `setInterval` stall probe; `perf_hooks.monitorEventLoopDelay` per phase; `node --cpu-prof --cpu-prof-interval 1000` (1 ms sampling).
Extract phase	`pdf-parse` (with `normaliseTableHorizonMarkers`) on reportlab-generated PDFs, plus cheerio on HTML.

`server/scripts/analyze-cpuprofile.ts` parses the `.cpuprofile`:

- Consecutive non-idle samples form a “busy run”. A busy run of **≥ 50 ms** counts as a blocking slice.
- Blocking time is attributed by: leaf frame; deepest app frame (`server/`); module; native or library; entry app frame.
- Concentration metrics: top-1/3/5 share, HHI over app functions, top-module share.
- Frames from the harness itself (`server/scripts/` , with no app frame beneath) are excluded and reported separately as `harnessExcludedMs` .

Commands (from the repo root; `DATABASE_URL` is a dummy and is never connected):

```
# 4 concurrent jobs, extract + analyze phases
DATABASE_URL=postgres://stub@127.0.0.1:1/stub PROFILE_OUT=/tmp/ciq-prof/c4/harness-
result.json \
  node --cpu-prof --cpu-prof-dir=/tmp/ciq-prof/c4 --cpu-prof-interval 1000 --import ts
x server/scripts/profile-worker-blocking.ts
# single-job control
DATABASE_URL=postgres://stub@127.0.0.1:1/stub PROFILE_CONCURRENCY=1
PROFILE_CORPORA=12 PROFILE_PHASES=analyze \
  PROFILE_OUT=/tmp/ciq-prof/c1/harness-result.json \
  node --cpu-prof --cpu-prof-dir=/tmp/ciq-prof/c1 --cpu-prof-interval 1000 --import ts
x server/scripts/profile-worker-blocking.ts
# attribution (main-thread profile = *.0.001.cpuprofile)
npx tsx server/scripts/analyze-cpuprofile.ts /tmp/ciq-prof/c4/CPU.*.0.001.cpuprofile /
tmp/ciq-prof/c4/analysis.json
```

The raw outputs are committed in `docs/worker-isolation/evidence/` : `harness-result-c4.json` , `harness-result-c1.json` , `cpuprofile-analysis-c4.json` and `cpuprofile-analysis-c1.json` .

2. Event-loop results

Run	Phase	Wall	ELD p50	ELD p99	ELD max	Stalls ≥ 50 ms (count / total)	Stalls > 1 s	Stalls > 5 s	RSS
c4 (4 jobs)	extract (pdf-parse/cheerio)	1.9 s	31 ms	65 ms	65 ms	—	0	0	—
c4 (4 jobs)	analyze (real pipeline)	69.5 s	10 ms	**534 ms**	**8,254 ms**	38 / **44.2 s**	9	2	**1,295 MB**
c1 (1 job, corpus 12)	analyze	36.1 s	10 ms	14 ms	**5,876 ms**	14 / **12.5 s**	1	1	876 MB

- With 4 concurrent jobs, the main thread was **blocked for 44 s out of 69.5 s (64 %)**. RSS is 1.3 GB, which matches production (~1.2 GB).
- A **single** job alone still blocks the loop for up to **5.9 s** at a time (12.5 s of 36 s). This is a property of the per-job pipeline, not only of concurrency. Concurrency stacks slices back to back, so the loop never catches up: p99 rose from 14 ms to 534 ms.
- An earlier repeat run (`/tmp/ciq-prof/run1.log`) reproduced this: ELD max 8,460 ms, 36 stalls / 48.3 s, RSS 1,288 MB.
- All 4 companies reached “Analysis complete” (scored measures). The final fenced score-write returned `Cancelled (score_write_fenced)` . This is expected: the pg stub returns 0 rows for the ownership-fenced `UPDATE … RETURNING` , so ownership is not confirmed. It is after all CPU work and does not affect the profile.

3. CPU-profile attribution (c4 main thread)

Profiled 74.5 s; busy 46.9 s; **37 blocking slices = 45.5 s**; longest slice **8,846 ms**. Harness frames excluded: 1.3 s.

3.1 Slice length distribution

Slice length	Runs	Total
50–100 ms	2	0.13 s
100–250 ms	3	0.49 s
250–500 ms	8	3.20 s
500–1000 ms	15	11.15 s
1–2 s	5	7.06 s
2–5 s	1	2.73 s
≥ 5 s	3	**20.76 s**

3.2 Hot frames (deepest app frame, share of blocking time)

#	Function (module)	ms	% of blocking
1	`tokenize` (passage-retrieval)	7,411	16.3
2	garbage collector (no app frame)	6,964	15.3
3	anonymous fn (corpus-boilerplate)	5,228	11.5
4	`detectSecHeadingInFragment` (passage-retrieval)	2,692	5.9
5	`bm25Score` (passage-retrieval)	2,287	5.0
6	anonymous fn (passage-retrieval)	2,118	4.7
7	`buildBM25Index` (passage-retrieval)	2,053	4.5
8	`chunkBody` (passage-retrieval)	1,995	4.4
9	anonymous fn in `summarizeDocuments` (analyzer.ts:222)	1,646	3.6
10	`scoreAboutness` (aboutness)	1,358	3.0
11	`applyCorpusHygiene` (corpus-hygiene)	1,211	2.7
12	`recoverSecHeadingNewlines` (passage-retrieval)	1,043	2.3
13	anonymous fn (processor)	1,028	2.3
14	`isLikelyToc` (passage-retrieval)	995	2.2
15		840	1.8

#	Function (module)	ms	% of blocking
	`computeAssemblyPenaltyFactors` (corpus-boilerplate)		
16	`words` (corpus-boilerplate)	805	1.8
17	`buildEvidencePackForMeasure` (passage-retrieval)	682	1.5
18	`countTopicHits` (passage-retrieval)	554	1.2
19	`scanDocumentsForWithdrawals` (temporal-validation)	441	1.0
20	`stripRepeatedStructuralBlocks` (corpus-boilerplate)	246	0.5
21	`looksNonEnglish` (translation)	236	0.5

Leaf natives: GC 15 %; regex natives (`\s+` ≈ 6 %, CJK range ≈ 2 %, `[^\w\s]` ≈ 2 %, TOC regex ≈ 1 %, topic-term regex ≈ 1 %). App code self-time is ≈ 52 %.

3.3 By module

Module	% of blocking
passage-retrieval	49.0
runtime / GC / no app frame	19.7
corpus-boilerplate	16.0
analyzer	4.3
aboutness	3.3
corpus-hygiene	2.7
processor	2.3
temporal-validation	1.0
passage-rescore	0.9
translation	0.6

Entry frames: `analyzeCompanyMeasures` 42.1 %, `summarizeDocuments` 32.7 %, `runAnalyzePhase` 4.3 %, `translateDocumentsToEnglish` 0.6 %.

3.4 Longest slices (what one multi-second block is made of)

Slice	Composition (top app frames)
8,846 ms	corpus-boilerplate anon 26 %, GC 19.7 %, `tokenize` 14.6 %
6,301 ms	corpus-boilerplate anon 23 %, GC 18.3 %, `summarizeDocuments` anon 12.9 %
5,614 ms	GC 21.8 %, `applyCorpusHygiene` 21.6 %, `scoreAboutness` 17.2 %
2,731 ms	corpus-boilerplate anon 26 %, GC 12 %, `summarizeDocuments` anon 10.8 %
1,766 ms	`tokenize` 26.6 %, `detectSecHeadingInFragment` 10.7 %, GC 10.5 %

3.5 Single-job control (c1)

Metric	Value
Blocking slices	17 = 13.5 s; longest 5,864 ms
Distinct app functions	152
Top-1	GC 15 %
`tokenize`	13 %
corpus-boilerplate anon	10 %
top-3 / top-5	38.1 % / 50 %
HHI	0.071
Top module (passage-retrieval)	44.1 %

The single-job profile has the same shape as c4, so the distribution is not an artefact of concurrency.

4. Verdict: DISTRIBUTED

Concentration metric (c4)	Value	Reading
Distinct app functions in blocking slices	312 (28 modules)	broad
Top-1 function share	16.3 %	no dominant hotspot
Top-3 / Top-5 share	43.1 % / 54.0 %	about half the time is outside the top 5
HHI (app functions)	0.081	unconcentrated (< 0.15)
GC share	15–20 %	belongs to the whole 1.3 GB heap, not to one function

The blocking is spread across **every analysis stage**:

- corpus hygiene;
- boilerplate stripping;
- document summarisation;
- SEC-heading recovery;
- chunking;
- BM25 indexing and scoring;
- aboutness;
- temporal withdrawal scan;

- evidence-pack assembly;
- language detection.

Each multi-second slice is itself a mix of 3+ stages plus GC. Moving any single function, or even the top 5, into a `worker_threads` pool would leave about half of the blocking time on the main loop. Multi-second slices from corpus-boilerplate, hygiene and aboutness would remain. The GC component cannot be moved at function granularity: it comes from the size of the heap that the per-job data structures create on the main isolate.

Conclusion: the **smallest boundary that removes the blocking is the per-job processor**. The whole `runAnalysisPipeline` for a job must run outside the BullMQ main loop, in a sandboxed child process. Phase 2 (DESIGN.md) designs that boundary.

5. Production extrapolation (hypothesis, not measured)

Corpus 12 (43 M chars, 50 MB JSON) produced a single 5.9 s slice when running alone on this sandbox. Production corpora reach 120–1,160 MB of JSON (for example var_corpus 13 = 294 MB, 102 = 484 MB, 16 = 1.16 GB). The work is roughly linear in corpus size: tokenize, boilerplate n-gram passes and BM25. If so, single slices of tens of seconds to minutes are expected on those companies, with 4 of them interleaved per replica and GC pressure growing with the heap.

That is consistent with the batch 1274 lag (p99 36–242 s) and with lock renewal being starved for ~1,250 s. This extrapolation is a **hypothesis**: it has not been measured on the production corpora or production CPUs.

6. Caveats

- The sandbox has 2 vCPU and 7 GB RAM. Production CPU speed differs, so absolute durations will differ; the *distribution* is the finding.
- LLM latency was stubbed at 400 ms. Real latency only changes how often the loop is idle between CPU bursts, not the length of a synchronous slice.
- The extract phase used synthetic reportlab PDFs. On those, `pdf-parse` was not a material blocker (ELD max 65 ms). Real scanned or very large PDFs were not profiled. Fetch, browser and network code paths were not exercised (zero-network).
- Line numbers in the profile are tsx-transpiled positions (many show `:1`). Function names and modules are reliable.
- The final score-write ended `Cancelled (score_write_fenced)` because of the pg stub (see §2). This is expected and happens after all CPU work.

7. Observed BM25 re-tokenisation and the evidence-validation loop (requested characterisation)

This section characterises the two loops called out in review. It is **evidence only**: no scoring-rule or BM25 change is made on this branch — the isolation fix moves the whole pipeline off the main loop and preserves scoring byte-for-byte. Optimising these loops is a separate, out-of-scope follow-up.

Figures are from the 4-concurrent capture (`evidence/cpuprofile-analysis-c4.json`): `totalProfiledMs` 74,530; `busyMs` 46,936 (**63 %**); `blockingMs` 45,524. Entry frames `analyzeCompanyMeasures` 19,184 ms and `summarizeDocuments` 14,892 ms dominate — i.e. the per-measure analysis path, not fetch/LLM.

7.1 BM25 re-tokenisation (redundant work, not a correctness bug)

`buildBM25Index(chunks)` (`passage-retrieval.ts:342`) tokenises **every chunk once** to populate `docFreqs` / `docLengths` , then stores the index as `{ documents: chunks, docFreqs, docLengths, … }` — it keeps the **raw chunk strings**, not the token arrays. `bm25Score(query, docIndex, index)` (`passage-retrieval.ts:377`) therefore re-runs `const docTokens = tokenize(index.documents[docIndex])` **on every scoring call**. Each chunk is thus tokenised once at index build and **again every time it is scored**.

Profile support (self-time, c4):

Frame	ms	note
`tokenize` (total / leaf)	**7,411 / 4,525**	top app function; 16.3 % of blocking
`RegExp: \s+` (native, inside `tokenize`)	2,679	the split that `tokenize` re-runs per re-tokenisation
`bm25Score`	2,287	caller that re-tokenises `index.documents[docIndex]`
`buildBM25Index`	2,053	the one *non-redundant* tokenise pass

The gap between `buildBM25Index` (2,053 ms, one pass over all chunks) and total `tokenize` (7,411 ms) is the re-tokenisation tax: ~3.6× the work of a single pass, driven almost entirely by `bm25Score` re-walking cached documents. A per-index token cache (tokenising each chunk once, reused by `bm25Score`) would remove most of it — but that is a scoring-path change and is deliberately **not** in this branch.

7.2 Evidence-validation loop (amplifier)

Evidence packs are built per measure: `measures.map((measure) => buildEvidencePackForMeasure({ measure, chunks, bm25Index, … }))` (`passage-retrieval.ts:2352`), 36 iterations per company. Each iteration (a) re-tokenises the measure's own keywords (`evidenceKeywordTokens.push(...tokenize(kw))` , `passage-retrieval.ts:1349`) and (b) scores candidate chunks through `bm25Score` , which re-tokenises those chunks again (7.1). The **same documents are re-tokenised across all 36 per-measure passes**, so §7.1's per-call cost is multiplied by the measure fan-out. This is why `analyzeCompanyMeasures` / `summarizeDocuments` carry ~34 s of the 45.5 s blocking total: the loop structure amplifies a redundant per-document cost across 36 measures × many chunks.

7.3 Why this supports the isolation decision (not a scoring fix)

Even if re-tokenisation were cached away, ~half the blocking time remains — GC (6,964 ms), `corpus-boilerplate` (7,268 ms module total), `chunkBody` , `scoreAboutness` , `applyCorpusHygiene` , SEC-heading regexes. The blocking is **distributed** (top-5 app functions ≈ 44 % of blocking; 312 distinct app functions), so no single hot-function fix unblocks the loop. The smallest boundary that reliably stops the pipeline blocking lock renewal is moving the **whole per-job pipeline** off the main loop (DESIGN.md). §7.1–§7.2 are recorded as a prioritised follow-up for a later, separately-reviewed scoring-performance change.